Booleans, Logic & Clean Code

Class 4 — In-Class Work-Along Sheet (working title · built iteratively)

Name: _____ Section: _____ Date: _____

How to use this sheet: Write your prediction **first**, then run it. A wrong guess you then fix sticks far better than a right answer handed to you. Keep your **Terminal** open throughout.

Part 1 — Comments: Notes to Your Future Self

Q1. *(Decode it — no hints, no running yet.)* Read this program. In **60 seconds**, what does it compute? Write your best guess — *struggling here is the point*:

```
a = 3
b = 4
c = 5
s = (a + b + c) / 2
area = (s * (s-a) * (s-b) * (s-c)) ** 0.5
print(area)
```

Your guess:

_____

Q2. Here is the **exact same** program with **one line added** at the top:

```
# Heron's formula: area of a triangle from its three side lengths
a = 3
b = 4
c = 5
s = (a + b + c) / 2
area = (s * (s-a) * (s-b) * (s-c)) ** 0.5
print(area)
```

(a) That first line, starting with `#`, is called a _____.

(b) Who is it written for — the **human** reading the code, or the **machine** running it? _____

(c) Run **both** programs. Does adding the comment change what gets **printed**? _____

Q3. *(How to write one.)* A `#` turns everything after it *on that line* into a comment. Run this:

```
x = 5            # an inline comment, after some code
# a whole line that is a comment
print(x)
```

(a) It prints _____. (b) The `#` parts are completely _____ by Python *(ignored / run)*.

Q4. *(The one rule — explain WHY, not WHAT.)* Two comments for the same line. Which is **useful**?

```
seconds = days * 86400    # A:  multiply days by 86400
seconds = days * 86400    # B:  86400 = number of seconds in one day
```

(a) More useful: _____ because _____

(b) Rewrite this **useless** comment into a useful one: `celsius = kelvin - 273 # subtract 273 from kelvin`
→ `# _____`

Q5. *(Commenting out — predict, then run.)* Predict the output, then run:

```
print("step 1")
# print("step 2")
print("step 3")
```

(a) Output: _____ (b) Why didn't `step 2` appear? _____

(c) Putting `#` in front of a line to switch it **off** without deleting it is hugely useful for _____ *(debugging / making code faster)*.

Q6. *(Multi-line comments? — predict, then run.)* Does Python have a `/* ... */`-style multi-line comment like C++/Java? Try this:

```
print("before")
"""
is this a multi-line comment?
"""
print("after")
```

(a) Output: _____

(b) It *looks* like a comment, but `"""..."""` is really a _____ — Python just builds it and throws it away. *(Its real job, "docstrings," comes later.)*

(c) So to comment **several lines at once**, the real shortcut lives in your **editor**: select the lines and press _____ *(Mac)* — it stamps `#` on every line.

Q7. *(Recall — books closed.)*

- A comment is written for the _____ *(human / machine)* and is _____ by Python.
- The golden rule of a good comment: explain the _____, not the _____.
- One practical use of `#` besides leaving notes: _____

Part 2 — Operators That Judge: Comparisons & the Boolean

Q8. *(A new kind of answer — predict, then run.)* Every operator so far gave you a **number**. Predict what this gives:

```
print(5 + 3)
print(7 > 3)
```

(a) `5 + 3` = _____ (b) `7 > 3` = _____ — is that second answer a **number**? _____

Q9. *(The six comparison operators — predict ALL, then run.)*

```
print(7 == 3)
print(7 != 3)
print(7 < 3)
print(7 > 3)
print(7 <= 7)
print(3 >= 7)
```

(a) Fill in: `==`_____ `!=`_____ `<`_____ `>`_____ `<=`_____ `>=`_____

(b) In words: `==` means _____, `!=` means _____, `<=` means _____.

Q10. *(What type is the answer? — predict, then run.)*

```
print(type(7 > 3))
```

(a) `type(7 > 3)` = _____

(b) You met this type in **Class 2** but never made one yourself. Comparisons are *where booleans come from* — every comparison produces a _____ *(`True` or `False`)*.

Q11. *(The trap that bites everyone — `=` vs `==`.)* These look alike but do opposite jobs:

```
x = 5          # line A
print(x == 5)  # line B
```

(a) Line A _____ the value 5 to `x` *(assigns / asks)*.

(b) Line B _____ whether `x` equals 5 *(assigns / asks)* → prints _____.

(c) One sentence: when do you use `=` and when `==`? _____

Q12. *(The gem — predict FIRST, you'll be surprised.)*

```
print(True == 1)
print(False == 0)
print(True + True)
print(False + 5)
```

(a) Predict all four: _____

(b) `True + True` is **2**?! Underneath, `True` is really the number _____ and `False` is _____.

(c) So `True + False + True + True` would be _____ — which is just **counting how many are `True`**.

Q13. *(Precedence — predict, then run.)*

```
print(2 + 3 < 4 + 1)
```

(a) Result: _____ (b) Why? Python does the _____ first (`2+3` and `4+1`), *then* compares `5 < 5`.

(c) If unsure of the order, the safe move is always to add _____.

Q14. *(Recall — books closed.)*

- A comparison operator compares two values and produces a _____ (`True`/`False`) — of type _____.
- `=` is for _____; `==` is for _____.
- Underneath, `True` is the number _____ and `False` is _____.

Part 3 — Combining Conditions: `and`, `or`, `not` (& Truthiness)

Q15. *(One comparison isn't enough — build it.)* A student **passes** if `score >= 40` **AND** `attendance >= 75`. A single comparison can't say both at once. Predict, then run (with `score = 55`, `attendance = 80`):

```
score = 55
attendance = 80
print(score >= 40 and attendance >= 75)
```

(a) Result: _____ (b) Change `attendance` to `60` and re-run → _____ (c) Why did it flip?

_____

Q16. *(The three logical operators — predict ALL, then run.)*

```
print(True and False)
print(True or False)
print(not True)
print((5 > 3) and (2 > 4))
```

(a) Fill: `and` _____ `or` _____ `not True` _____ last line _____

(b) In words: `and` is `True` only when _____; `or` is `True` when _____; `not` _____ the value.

Q17. *(Truthiness — predict, then run.)* Every value is secretly "truthy" or "falsy". Predict each:

```
print(bool(0), bool(0.0), bool(""), bool(None))
print(bool(5), bool("hi"), bool(-1))
```

(a) The **falsy** values here are: _____

(b) The rule of thumb: `0`, empty text `""`, and `None` are _____; pretty much **everything else** is _____.

Q18. *(The gem — predict FIRST, this one is wild.)* `or` and `and` don't always return `True` / `False` — predict each, then run:

```
print(0 or 5)
print("" or "hi")
print(3 and 7)
print(0 and 7)
```

(a) Predict all four: _____

(b) `0 or 5` is **5**, not `True`?! Rule: `or` returns the **first** _____ **value** (else the last); `and` returns the **first** _____ **value** (else the last).

Q19. *(Why that's useful — the default-value trick.)* Predict, then run:

```
name = ""
print(name or "Guest")
name = "Asha"
print(name or "Guest")
```

(a) First print: _____ second print: _____

(b) In one line, what real-world job does `name or "Guest"` do? _____

Q20. *(Precedence — predict, then run.)*

```
print(True or False and False)
```

(a) Result: _____ (b) Order among the three: `not` is strongest, then _____, then _____.

(c) To avoid all doubt, the safe move is still to add _____.

Q21. *(Recall — books closed.)*

- `and` → `True` only if _____; `or` → `True` if _____; `not` → _____.
- A value used as a condition is either _____ or _____ (e.g. `0` / `""` / `None` are _____).
- Precedence order of the three, strongest first: _____ → _____ → _____.

Part 4 — Debugging Lab: Operators in the Wild

Q22. *(This code is broken. Your job is to find out why.)*

You are looking at an excerpt from `driver_dispatch.py` — a dispatch module at a ride-sharing startup. The **business rules are written in the comments** at the top; the code is supposed to implement them exactly. Find every bug.

```
# ── driver_dispatch.py  (RideSurge Technologies · Core Dispatch Module) ──
# Business rules (Ops Manual v5.1):
#   Rule A:  Driver can accept a PREMIUM ride ONLY IF all three hold:
#              rating >= 4.7  AND  trips_completed >= 200  AND  car_type == "sedan"
#   Rule B:  SURGE pricing fires when the rider/driver ratio is AT LEAST 3 (>= 3)
#              AND it is currently raining.
#   Rule C:  FREE-RIDE coupon issued when customer is a new user AND at least one
#              of: (referral code used) OR (promo is active).
#              Existing users are NOT eligible, even if a promo is running.

driver_rating     = 4.8
trips_completed   = 150       # below the required 200-trip threshold
car_type          = "sedan"

active_riders     = 114       # exactly 3 times active_drivers
active_drivers    = 38
rain              = True

is_new_user       = False     # this is a returning customer, not a new one
referral_code     = ""
promo_active      = True

# Rule A — Premium ride eligibility
can_take_premium = driver_rating >= 4.7 or trips_completed >= 200 or car_type == "sedan"

# Rule B — Surge pricing
surge_active = (active_riders / active_drivers) > 3 and rain == True

# Rule C — Free-ride coupon
free_ride_coupon = is_new_user and referral_code != "" or promo_active

print("Premium eligible:", can_take_premium)
print("Surge active:    ", surge_active)
print("Free ride coupon:", free_ride_coupon)
```

(a) Run it. Write the three outputs you see:

```
Premium eligible: ________
Surge active:     ________
Free ride coupon: ________
```

(b) Read the business rules and the variable values carefully. Write what each output **should** be — what the rules actually require:

```
Premium eligible: ________
Surge active:     ________
Free ride coupon: ________
```

(c) This code has **3 bugs** — each is one operator used incorrectly. Find all three. For each, write which Rule line has the problem, what the wrong operator is, and what it should be:

Bug 1:

_____

Bug 2:

_____

Bug 3:

_____

(d) Write the corrected version of each buggy line:

```
can_take_premium = ____________________________________________________
surge_active     = ____________________________________________________
free_ride_coupon = ____________________________________________________
```

Part 5 — When `+` Isn't Addition: Operators Across Types

Q23. *(The hook — predict FIRST, no running yet.)* You have used `+` to add numbers since Class 1. Predict what these two lines print:

```
print(2 + 3)
print("2" + "3")
```

(a) Predict: `2 + 3` → _____ `"2" + "3"` → _____

(b) Run both. If `"2" + "3"` surprised you — in one line, what did `+` do to the two strings?

_____

Q24. *(String `+` and `*` — predict ALL, then run.)*

```
print("Hello" + ", " + "World!")
print("ha" * 3)
print("-" * 30)
print(type("ha" * 3))
```

(a) Predict all four: _____

(b) In words: `+` on strings _____ them; `*` on a string _____ it.

(c) `"-" * 30` is a handy trick to print a _____ without typing 30 dashes yourself.

Q25. *(What about `-` and `/`? — predict, then run.)*

```
print("hello" - "ello")
print("hello" / 2)
```

(a) Predict both outputs: _____

(b) Run both. What error type appears? _____

(c) `+` and `*` work on strings; `-` and `/` don't. In one line, why do you think that is? _____

Q26. *(Type-mixing — predict FIRST, this one is interesting.)*

```
print("123" + 4)
print("123" == 4)
```

(a) Predict both: _____

(b) Run both. One crashes; one doesn't. Fill in: `"123" == 4` → _____ `"123" + 4` → _____

(c) Why the difference? `==` just asks **"are these the same thing?"** — no conversion needed, the answer is simply _____. But `+` would have to *choose*: does it mean `"1234"` (join as text) or `127` (add as numbers)? Python _____ *(picks one silently / refuses and asks you to be explicit).*

(d) Fix the `+` — write both ways:

- Treat `4` as text → `print("123" + _____)`
- Treat `"123"` as a number → `print(_____ + 4)`

(e) *(Design question.)* JavaScript silently treats `4` as text and gives `"1234"` for `"123" + 4`. Why is Python's approach safer? _____

Q27. *(Operator overloading — the principle, then your own research.)*

(a) Fill in: Python uses the _____ of the operands to decide what `+` *does*. `int` tells Python "`+` means _____"; `str` tells Python "`+` means _____". Same symbol, each type has its own rulebook. This is called **operator overloading**.

(b) *Mini research:* Open your terminal and try these — write the outputs:

```
print([1, 2] + [3, 4])
print([1, 2] * 3)
```

`[1, 2] + [3, 4]` → _____ `[1, 2] * 3` → _____

(c) For **lists**, `+` means _____ and `*` means _____. Is this consistent with what `+` and `*` do to strings? _____

(d) *(Optional — go further.)* Search "Python operator overloading examples" and find one more type (other than `int`, `str`, or `list`) where `+` does something interesting. Write it here: _____

Q28. *(Recall — books closed.)*

- `+` on two strings → _____; `*` on a string and a number → _____.
- `"hi" - "h"` gives a _____ error because _____.
- `"5" + 3` raises a _____ error; the two explicit fixes: _____ or _____.
- `"5" == 3` gives _____ with no error — `==` only checks _____, needs no conversion.
- The name for "each type defines its own meaning for `+`": _____.

Part 6 — The Language of Bits: Binary Numbers

Q29. *(The hook — run this first, then answer.)*

```
print(bin(13))
print(bin(255))
```

(a) `bin(13)` prints _____ `bin(255)` prints _____

(b) Both outputs start with `0b`. What do you think the `b` stands for? _____

(c) The digits after `0b` only use `0` and `1`. Why can't there be a `2` or `3`? _____

Q30. *(Base 10 — what you already know.)*

Every digit in a decimal number has a **position** worth a power of 10:

```
347  =  3 × 100  +  4 × 10  +  7 × 1
     =  3 × 10²  +  4 × 10¹  +  7 × 10⁰
```

(a) Write `529` as a sum of powers of 10: `529 = ____ × 10² + ____ × 10¹ + ____ × 10⁰`

(b) We call this "base 10" because each position is a power of _____, and digits can be 0–_____.

(c) Binary is "base 2": each position is a power of _____, and digits can only be _____ or _____.

Q31. *(Binary → decimal: reading a binary number.)*

The binary number `1101` means:

```
1101  =  1 × 2³  +  1 × 2²  +  0 × 2¹  +  1 × 2⁰
      =  1 × 8   +  1 × 4   +  0 × 2   +  1 × 1
      =  8 + 4 + 0 + 1
      =  13
```

(a) Convert `1010` to decimal (show your working):

```
1010  =  1 × 2³ + 0 × 2² + 1 × 2¹ + 0 × 2⁰
      =  ____  + ____  + ____  + ____
      =  ________
```

(b) Convert `11111` to decimal: _____

(c) Verify both answers in Python:

```
print(int("1010", 2))
print(int("11111", 2))
```

Results: _____ and _____ Do they match your hand answers? _____

Q32. *(Decimal → binary: the algorithm.)*

To convert a decimal number to binary, **repeatedly divide by 2 and collect the remainders** — then read them bottom to top:

```
13 ÷ 2 = 6  remainder 1   <-- LSB (last digit)
 6 ÷ 2 = 3  remainder 0
 3 ÷ 2 = 1  remainder 1
 1 ÷ 2 = 0  remainder 1   <-- MSB (first digit)

Read remainders bottom to top:  1 1 0 1  =  1101
```

(a) Use the algorithm to convert `22` to binary. Show each division step:

```
22 ÷ 2 = ____  remainder ____
__ ÷ 2 = ____  remainder ____
__ ÷ 2 = ____  remainder ____
__ ÷ 2 = ____  remainder ____
__ ÷ 2 = ____  remainder ____
Binary: ________
```

(b) Verify: `print(bin(22))` → _____ Does `0b` followed by your answer match? _____

(c) Convert `100` to binary by hand: _____ Verify: `print(bin(100))` → _____

Q33. *(Powers of 2 — the landmarks.)*

Fill in this table from memory (no calculator), then run `print(2**n)` to check:

```
+-----+--------+
| 2^0 |   1    |
| 2^1 |        |
| 2^2 |        |
| 2^3 |        |
| 2^4 |        |
| 2^5 |        |
| 2^6 |        |
| 2^7 |        |
| 2^8 |        |
+-----+--------+
```

Why is knowing these useful when reading binary numbers? _____

Q34. *(Python's base tools.)*

Python can show any integer in binary, octal, or hex — and convert back:

```
print(bin(42))          # base 2
print(oct(42))          # base 8
print(hex(42))          # base 16
print(int("101010", 2)) # binary string -> decimal
print(int("52", 8))     # octal string  -> decimal
print(int("2a", 16))    # hex string    -> decimal
```

(a) Fill in all six outputs: _____ · _____ · _____ · _____ · _____ · _____

(b) All three give the number _____ — same value, three different representations. The value itself doesn't change; only the _____ does.

(c) You can also write integer literals directly in other bases:

```
print(0b101010)   # binary literal
print(0o52)       # octal literal
print(0x2a)       # hex literal
```

All three print _____. The prefixes `0b` / `0o` / `0x` tell Python which base you mean.

Q35. *(Recall — books closed.)*

- In base 2, each position is a power of _____, and only digits _____ and _____ are allowed.
- To convert binary → decimal: sum of _____.
- The repeated-division algorithm for decimal → binary: divide by 2, collect remainders, read _____.
- `bin(n)` returns a _____ starting with `_____`; `int("1101", 2)` converts back to _____.

Part 7 — Bitwise Operators: Working on Individual Bits

Q36. *(The hook — predict FIRST.)*

```
print(13 & 10)
```

(a) Predict: `13 & 10` → _____

(b) Run it. Result: _____ — were you right?

(c) Now run this to see why:

```
print(bin(13))
print(bin(10))
print(bin(13 & 10))
```

Fill in:

```
  1101   (13)
& 1010   (10)
------
  ????
```

The rule: `&` compares each column — the result bit is `1` only when **both** bits are `1`.

Q37. *(`&` — bitwise AND.)*

Single-bit rule: `0 & 0` = _____ `0 & 1` = _____ `1 & 0` = _____ `1 & 1` = _____

Predict both, then run:

```
print(12 & 10)
print(6 & 3)
```

Show your column working for `12 & 10`:

```
  1100   (12)
& 1010   (10)
------
  ????
```

`12 & 10` → _____ `6 & 3` → _____ Verify with `bin()`: _____ · _____

Q38. *(`|` — bitwise OR.)*

Single-bit rule: `0 | 0` = _____ `0 | 1` = _____ `1 | 0` = _____ `1 | 1` = _____

Predict both, then run:

```
print(12 | 10)
print(6 | 3)
```

Show column working for `12 | 10`:

```
  1100   (12)
| 1010   (10)
------
  ????
```

`12 | 10` → _____ `6 | 3` → _____ Verify with `bin()`: _____ · _____

Q39. *(`^` — bitwise XOR: "exclusively one")*

XOR means **exactly one** of the two bits is `1` — same bits give `0`, different bits give `1`.

Single-bit rule: `0 ^ 0` = _____ `0 ^ 1` = _____ `1 ^ 0` = _____ `1 ^ 1` = _____

Predict, then run:

```
print(12 ^ 10)
print(6 ^ 3)
print(9 ^ 9)
print(9 ^ 0)
```

(a) Fill in all four: _____ · _____ · _____ · _____

(b) `9 ^ 9` = _____ — in words: anything XORed with itself gives _____.

(c) `9 ^ 0` = _____ — in words: anything XORed with zero gives _____.

(d) Why is XOR used in **encryption**? *(Think: if `a ^ key = secret`, what does `secret ^ key` give?)*

_____

Q40. *(Shifts — `<<` and `>>`)*

`<<` shifts all bits **left** by n positions (fills with 0 on the right); `>>` shifts **right**.

```
print(bin(3))       # 0b11
print(bin(3 << 1))  # shift left 1
print(bin(3 << 2))  # shift left 2
print(bin(3 << 3))  # shift left 3
```

(a) Fill in the binary results: _____ · _____ · _____ · _____

(b) What decimal values are these? `3<<1` = _____ `3<<2` = _____ `3<<3` = _____

(c) Pattern: each left-shift by 1 _____ the number. Shifting left by `k` = multiplying by _____.

```
print(16 >> 1)
print(16 >> 2)
print(16 >> 3)
print(7 >> 1)     # what happens with an odd number?
```

(d) `16>>1` = _____ `16>>2` = _____ `16>>3` = _____ `7>>1` = _____

(e) Each right-shift by 1 _____ the number. It behaves like _____ division (rounds down).

Q41. *(`&` / `|` vs `and` / `or` — the critical distinction.)*

These look similar but do completely different things. Predict all four, then run:

```
print(5 and 3)
print(5 & 3)
print(5 or 3)
print(5 | 3)
```

(a) Predict all four: _____ · _____ · _____ · _____

(b) Run them. Fill in:

- `5 and 3` = _____ — returns an _____ (logical operator, operand-return)
- `5 & 3` = _____ — returns an _____ (bitwise, column-by-column)

- `5 or 3` = _____ — returns the first _____ value
- `5 | 3` = _____ — `101 | 011` = _____ in binary = _____ decimal

(c) `5 or 3` and `5 | 3` gave different answers (_____ vs _____). In one line, why? _____

(d) When should you use `and` / `or` vs `&` / `|`? _____

Q42. *(Practical uses — applying what you know.)*

(a) Odd/even check:

```
print(7 & 1)
print(8 & 1)
```

`7 & 1` = _____ `8 & 1` = _____

Why does `n & 1` tell you whether `n` is odd or even? *(Hint: look at the last bit of any odd number.)*

_____

Is this equivalent to `n % 2`? _____

(b) Bit flags — permissions in one integer:

```
READ    = 0b100   # 4
WRITE   = 0b010   # 2
EXECUTE = 0b001   # 1

perms = READ | WRITE   # user can read and write, but not execute
print(bin(perms))

print(bool(perms & READ))      # can the user read?
print(bool(perms & EXECUTE))   # can the user execute?
```

`bin(perms)` = _____

`perms & READ` → _____ (non-zero means the bit is _____)

`perms & EXECUTE` → _____ (zero means the bit is _____)

(c) To grant execute permission: `perms = perms | _____` To check all three: `perms == (READ | WRITE | EXECUTE)` gives _____ before granting and _____ after.

Q43. *(Recall — books closed.)*

- `&` gives `1` only when _____; `|` gives `1` when _____; `^` gives `1` when _____.
- `n << k` multiplies `n` by _____; `n >> k` divides `n` by _____ (rounding _____).
- `n & 1 == 1` means `n` is _____; `n & 1 == 0` means `n` is _____.
- Key difference: `and` / `or` return _____ and may short-circuit; `&` / `|` work _____ and always return an _____.

Part 8 — Float Precision: Why `0.1 + 0.2 ≠ 0.3`

Q44. *(The hook — predict FIRST.)*

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

(a) Predict both: _____ _____

(b) Run them. What did you actually get? _____ _____

(c) Were you surprised? This is not a Python bug — it happens in **every** language that uses the same storage format. Your job in this part is to understand exactly why.

Q45. *(Finite memory → finite precision.)*

Python's `float` is stored in exactly **64 bits** — always, for every float.

(a) 64 bits means at most `2^64` ≈ _____ distinct values can ever be represented.

(b) But how many real numbers exist between `0.0` and `1.0`? _____

(c) So most decimals can only be stored as the _____ representable value — an approximation.

(d) Analogy: if you could only write **3 decimal digits**, `1/3` becomes:

```
print(f"{1/3:.3f}")    # output: ________
```

That's not the exact value — it's the best 3-digit approximation. Python's floats are the same idea, but with 64 bits instead of 3 decimal digits.

Q46. *(Why `0.1` is special: binary fractions.)*

To convert a decimal fraction to binary, **multiply by 2 repeatedly** and collect the integer parts (same spirit as the decimal→binary algorithm from Part 6, but for the fractional side):

Fill in the table for `0.1` — continue until you see the pattern repeat:

```
+--------+--------+-----------+-------+
| Step   | Value  | × 2       | Digit |
+--------+--------+-----------+-------+
|   1    |  0.1   |   0.2     |   0   |
|   2    |  0.2   |   0.4     |   0   |
|   3    |  0.4   |   0.8     |   0   |
|   4    |  0.8   |   1.6     |   1   |
|   5    |  0.6   |   1.2     |   1   |
|   6    |  0.2   |  ______   | ___   |  ← same as step 2!
|   7    |        |  ______   | ___   |
|   8    |        |  ______   | ___   |
|   9    |        |  ______   | ___   |
+--------+--------+-----------+-------+
```

(a) `0.1` in binary = `0.0001` _____ _____ _____ … *(the pattern repeats forever)*

(b) Just like `1/3 = 0.333...` repeats in base 10 because 3 doesn't divide into 10 evenly — `0.1` repeats in base 2 because _____ doesn't divide into _____ evenly.

(c) Python has to cut the repeating pattern off at bit 52. Run this to see the actual stored value:

```
print(f"{0.1:.55f}")
```

Output: _____

(d) Is the stored value slightly **above** or **below** `0.1`? _____

Q47. *(How Python stores a float: IEEE 754 — three fields in 64 bits.)*

Every Python float uses the **IEEE 754 double-precision** format — the same standard used by virtually all modern hardware and languages:

```
+--------+--------------------+------------------------------------------+
| 1 bit  |      11 bits       |                 52 bits                  |
|  sign  |      exponent      |                mantissa                  |
+--------+--------------------+------------------------------------------+
```

- **Sign:** `0` = positive, `1` = negative.
- **Exponent:** which power of 2 to scale by (like the `10^n` part of scientific notation).
- **Mantissa:** the significant digits after the leading `1` (the `1.xxxxx` part).

The full value = `(-1)^sign × 1.mantissa × 2^(exponent-1023)`

This is binary scientific notation: `2.56 ≈ 1.0100... × 2¹`

(a) Run this to see Python's float limits:

```
import sys
print(sys.float_info.dig)     # significant decimal digits of precision
print(sys.float_info.max)     # largest representable float
```

Precision: _____ decimal digits. Max float ≈ _____

(b) `sys.float_info.dig = 15` means Python floats are reliable to about _____ significant decimal digits — beyond that, you're seeing the approximation error.

(c) Now run to see all four stored values side by side:

```
print(f"{0.1:.20f}")
print(f"{0.2:.20f}")
print(f"{0.3:.20f}")
print(f"{0.1 + 0.2:.20f}")
```

Fill in the first 5 significant digits of each:

- stored `0.1` ≈ `0.1000` _____
- stored `0.2` ≈ `0.2000` _____
- stored `0.3` ≈ `0.2999` _____
- stored `0.1+0.2` ≈ `0.3000` _____

(d) `0.1` and `0.2` are stored slightly _____ their true values; `0.3` is stored slightly _____ its true value — so their sum doesn't match the stored `0.3`.

Q48. *(The practical rule: never `==` on floats — and how to cope.)*

(a) Run this — which comparisons work?

```
print(0.1 + 0.2 == 0.3)            # direct ==
print(round(0.1 + 0.2, 1) == 0.3)  # round first
import math
print(math.isclose(0.1 + 0.2, 0.3))  # tolerance-based
```

Results: _____ · _____ · _____

(b) `math.isclose(a, b)` returns `True` if `a` and `b` are within a tiny **tolerance** of each other. Why is this better than `==` for floats? _____

(c) For display, f-strings round automatically:

```
print(f"{0.1 + 0.2:.1f}")
```

Output: _____ — the stored noise disappears when you only ask for 1 decimal place.

(d) Complete the rules:

- For display: use _____ formatting or `round()`.
- For comparison: use `math._____()` instead of `==`.
- The root cause: `0.1` in binary is a _____ fraction — Python cuts it off at bit _____, leaving a tiny error.

Q49. *(Recall — books closed.)*

- Python's `float` uses exactly _____ bits, giving about _____ significant decimal digits of precision.
- `0.1` cannot be stored exactly because in binary it is a _____ fraction.
- The three fields of an IEEE 754 float: _____ (1 bit) · _____ (11 bits) · _____ (52 bits).
- Never use _____ to compare floats; use `math._____()` instead.

Part 9 — Operator Precedence: The Full Picture

Q50. *(Reconstruct the ladder from memory — before checking anything.)*

(a) These six priority groups span Classes 3 and 4. Number them **1 (highest) to 6 (lowest)** from memory:

```
___ not
___ **
___ <<  >>
___ and
___ or
___ ==  !=  <  >  <=  >=
```

(b) The three bitwise operators sit **between shifts and comparisons** — tightest first. Fill in what goes in each slot:

```
<<  >>          (shifts)
___ bitwise ___
___ bitwise ___
___ bitwise ___
==  !=  ...     (comparisons)
```

(c) Check your answers against the class table. How many of the 9 rows correct? ____/9

Q51. *(Footgun 1: `not` vs comparison — PREDICT before running.)*

`==` has **higher** precedence than `not`.

(a) So Python parses `not 2 == 1` as: `not (2 == 1)` **or** `(not 2) == 1`? _____

(b) Predict both outputs before running:

```
print(not 2 == 1)
print((not 2) == 1)
```

Predictions: _____ · _____

(c) Run both. Results: _____ · _____

(d) Trace the first expression step by step:

```
not 2 == 1
  step 1: 2 == 1   →  _______
  step 2: not ___  →  _______
```

(e) If you actually want `(not 2) == 1`, you **must** write: `print(_____)` *(adding explicit brackets).*

Q52. *(Footgun 2: shift vs arithmetic — PREDICT before running.)*

`+` has **higher** precedence than `<<`.

(a) So Python parses `3 << 1 + 1` as: `3 << (1+1)` **or** `(3<<1) + 1`? _____

(b) Predict both:

```
print(3 << 1 + 1)
print((3 << 1) + 1)
```

Predictions: _____ · _____

(c) Run both. Results: _____ · _____

(d) Trace the first:

```
3 << 1 + 1
  step 1: 1 + 1      →  ____
  step 2: 3 << ____  →  ____
```

Q53. *(Footgun 3: bitwise vs comparison — PREDICT before running.)*

`&` has **higher** precedence than `==`.

(a) So Python parses `12 & 10 == 8` as: `(12 & 10) == 8` **or** `12 & (10 == 8)`? _____

(b) Predict both:

```
print(12 & 10 == 8)
print(12 & (10 == 8))
```

Predictions: _____ · _____

(c) Run both. Results: _____ · _____

(d) Trace both:

```
12 & 10 == 8:
  step 1: 12 & 10     →  ____   (1100 & 1010 = ____)
  step 2: ____ == 8   →  ____

12 & (10 == 8):
  step 1: 10 == 8     →  ____
  step 2: 12 & ____   →  ____
```

(e) The two results are `True` vs `0`. Both happen to be falsy/truthy in the same direction — why might the type difference (`bool` vs `int`) still cause a bug? _____

Q54. *(All layers at once — predict every step, then run.)*

Trace `not 2 > 1 and 5 | 2 == 7` using the full precedence ladder:

```
not 2 > 1 and 5 | 2 == 7

step 1  ( | ):  5 | 2      →  ____
         now:  not 2 > 1 and ____ == 7

step 2  ( > ):  2 > 1      →  ____
step 3  (== ):  7 == 7     →  ____
         now:  not ____ and ____

step 4  (not):  not ____   →  ____
         now:  ____ and ____

step 5  (and):  ____ and ____  →  ____
```

Your prediction: `print(not 2 > 1 and 5 | 2 == 7)` → _____

Run it. Match? _____

Q55. *(Recall + the golden rule — books closed.)*

(a) Rank these five operators: **1 = highest precedence, 5 = lowest:**

`or` `**` `not` `<<` `==`

```
___ **     ___ <<     ___ ==     ___ not     ___ or
```

(b) The golden rule: **bracket, don't memorise**. Rewrite each with explicit brackets:

- `not x == y` → _____
- `3 << 1 + 1` → _____
- `a or b and c` → _____

(c) In one sentence, why does bracketing matter when multiple people work on the same codebase?

_____